CloudWalk – Monitoring Analyst Test

Transaction Monitoring and Anomaly Detection Analysis

By Eduardo Adão on 2026-03-14

---

1. Introduction

This document presents the analysis and monitoring proposal developed for the Monitoring Analyst Test.

The challenge consists of two parts:

1. Analysis of checkout behavior using historical comparison datasets (checkout_1.csv and checkout_2.csv).
2. Design of a monitoring approach capable of detecting anomalies in transaction behavior using transactions.csv and transactions_auth_codes.csv.

The methodology used in this analysis combines:

- historical baseline comparison
- SQL-based aggregation
- statistical distribution analysis
- rate-based anomaly detection
- rule-based monitoring thresholds

The objective is to identify abnormal system behavior and design a monitoring approach capable of detecting operational anomalies in near real time.

---

2. Additional Materials

The full exploratory analysis of the datasets was performed in a Google Sheets spreadsheet containing:

- data import of all CSV files
- intermediate calculations
- statistical summaries
- visualizations used during the analysis

The spreadsheet can be accessed at the following link:
CloudWalk – Monitoring Analyst Test
This material is provided as a supplementary reference to the analysis presented in this document.

---

3. Problem 1 – Checkout Behavior Analysis

3.1 Dataset Description

Two datasets were provided:

- checkout_1.csv
- checkout_2.csv

Each dataset contains hourly sales information with the following fields:

Column	Description
time	Hour of the day
today	Number of sales in the current day
yesterday	Sales at the same hour yesterday
same_day_last_week	Sales at the same hour in the previous week
avg_last_week	Average sales during the previous week
avg_last_month	Average sales during the previous month

This structure allows the current behavior to be compared with multiple historical references.

3.2 SQL Query Used for Analysis

Even though the data was provided as CSV files, the analysis was structured using SQL logic to simulate how this data would be queried in a monitoring environment.

The query below calculates a historical baseline and the deviation from that baseline.

```
SELECT
    time,
    today,
    yesterday,
    same_day_last_week,
    avg_last_week,
    avg_last_month,

    (same_day_last_week + avg_last_week + avg_last_month) / 3.0 AS baseline,

    today - ((same_day_last_week + avg_last_week + avg_last_month) / 3.0) AS
absolute_deviation,

    CASE
        WHEN (same_day_last_week + avg_last_week + avg_last_month) = 0
        THEN NULL
```

```
        ELSE
            (today - ((same_day_last_week + avg_last_week + avg_last_month) / 3.0))
            /
            ((same_day_last_week + avg_last_week + avg_last_month) / 3.0)
    END AS deviation_percentage

FROM checkout_1
-- FROM checkout_2
ORDER BY time;
```

The purpose of this query is to create a **historical expectation** for each hour and compare the current day behavior against that expectation.

This approach helps establish a historical reference for each hour, making it easier to visually identify deviations between current behavior and expected patterns.

3.3 Graphical Analysis

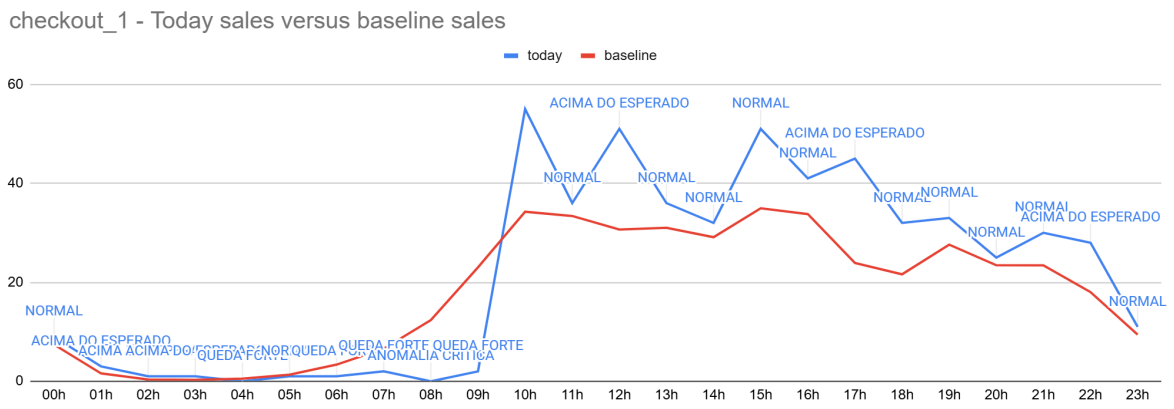
To visualize the anomaly behavior, the SQL output was exported to Google Sheets and plotted as a time-series graph.

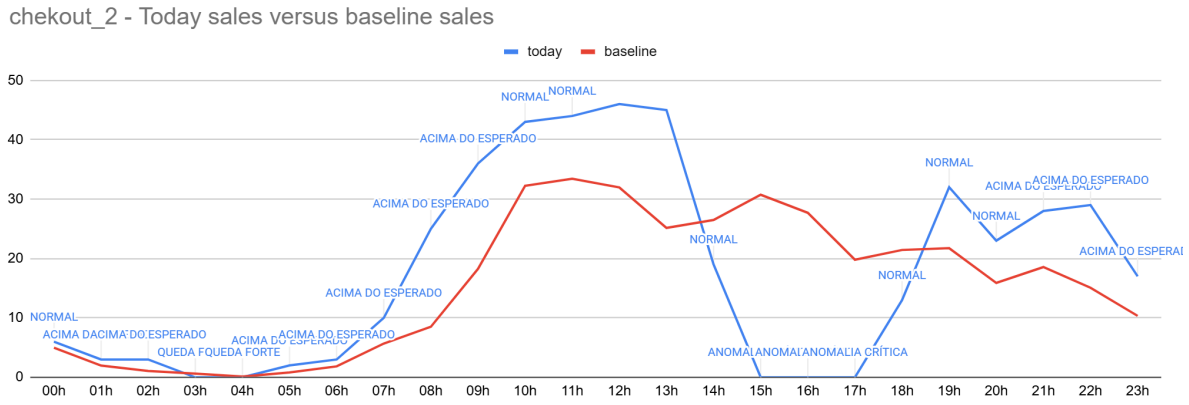
The graph displays:

- today
- baseline

This visualization focuses on identifying deviations between current behavior and historical expectations.

The objective of the visualization is not to establish monitoring thresholds, but rather to **clearly illustrate where the current system behavior diverges from the expected pattern**.





In a production monitoring environment, it would also be common to display upper and lower bounds around the baseline to detect anomalies automatically. However, for the purpose of this analysis the comparison between today and the historical baseline is sufficient to highlight abnormal behavior.

3.4 Analysis of checkout_1.csv

When looking at aggregated daily totals, the system appears stable.

Metric	Value
Today total	526
Yesterday total	523
Same day last week	572

The total transaction volume is therefore within historical expectations.

However, when analyzing the data at the hourly level, a clear anomaly appears in the early morning.

Critical observations include:

Hour	Today	Baseline	Deviation
08:00	0	~12.38	-100%
09:00	2	~23.02	-91%

These values represent extremely large deviations relative to the expected baseline.

Such a drop cannot reasonably be attributed to normal variation.

However, after 10:00 the system shows strong recovery and transaction volume even exceeds the historical baseline in several hours.

This pattern suggests:

- a temporary service degradation or interruption in the early morning
- followed by recovery
- possible demand compensation later in the day

Because the daily total remains close to the previous day, the incident appears to have been short-lived.

3.5 Analysis of checkout_2.csv

The second dataset shows a more severe anomaly.

During the morning and early afternoon, transaction volume is frequently above the historical baseline.

However, a critical deviation occurs between:

15:00 and 17:00.

During these hours:

Hour	Today	Baseline
15:00	0	~30
16:00	0	~27
17:00	0	~19

Three consecutive hours with zero transactions during a historically active period strongly indicate a system interruption.

After 18:00 the transaction volume resumes and eventually exceeds the historical baseline, suggesting system recovery.

However, the total daily volume remains below the previous day, indicating that the interruption had a measurable impact on the system's throughput.

3.6 Checkout Analysis Conclusion

Two different operational behaviors were identified.

checkout_1.csv indicates a temporary degradation in the early morning followed by recovery.

checkout_2.csv shows a stronger anomaly consistent with a partial outage lasting several hours.

This analysis highlights the importance of monitoring transactional systems at a fine temporal resolution, as aggregated daily metrics may hide short but impactful disruptions.

---

4. Problem 2 – Transaction Monitoring System

4.1 Dataset Description

Two datasets were provided:

transactions.csv

Column	Description
timestamp	minute-level timestamp
status	transaction status
count	number of transactions

Statuses include:

- approved
- denied
- failed
- reversed
- backend_reversed
- refunded

transactions_auth_codes.csv

Column	Description
timestamp	minute-level timestamp
auth_code	authorization code
count	number of occurrences

The dataset contains three authorization codes:

- 0
- 51
- 59

By comparing the totals between the two datasets, it is possible to observe that the counts of auth_code 51 and auth_code 59 together match the total number of denied transactions.

This suggests that these authorization codes are associated with denial events, although the precise meaning of each code cannot be inferred from the dataset alone.

4.2 Data Aggregation Strategy

The first step in the monitoring design was to aggregate transactions by minute and pivot status values into columns.

This allows transaction rates to be calculated and monitored over time.

4.3 SQL Query for Monitoring Dataset

The following SQL query performs the full monitoring preparation using CTEs.

It:

1. aggregates transactions by minute
2. calculates total transaction volume
3. computes rate metrics
4. calculates statistical baselines
5. exposes a time-series dataset ready for visualization

```
WITH status_pivot AS (

SELECT
timestamp,

SUM(CASE WHEN status='approved' THEN count ELSE 0 END) AS approved,
SUM(CASE WHEN status='denied' THEN count ELSE 0 END) AS denied,
SUM(CASE WHEN status='failed' THEN count ELSE 0 END) AS failed,
SUM(CASE WHEN status='reversed' THEN count ELSE 0 END) AS reversed,
SUM(CASE WHEN status='backend_reversed' THEN count ELSE 0 END) AS backend_reversed,
SUM(CASE WHEN status='refunded' THEN count ELSE 0 END) AS refunded

FROM transactions
GROUP BY timestamp

),

totals AS (
```

```sql
SELECT
*,
approved + denied + failed + reversed + backend_reversed + refunded AS total

FROM status_pivot

),

rates AS (

SELECT
timestamp,
approved,
denied,
failed,
reversed,
backend_reversed,
refunded,
total,

(denied / total) AS deny_rate,
(failed / total) AS fail_rate,
(reversed / total) AS reverse_rate

FROM totals

),

statistics AS (

SELECT
AVG(deny_rate) AS deny_mean,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY deny_rate) AS deny_median,
PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY deny_rate) AS deny_p95,
PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY deny_rate) AS deny_p99,

AVG(fail_rate) AS fail_mean,
PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY fail_rate) AS fail_p95,

AVG(reverse_rate) AS reverse_mean,
PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY reverse_rate) AS reverse_p95

FROM rates

)

SELECT
r.*,
s.*

FROM rates r
CROSS JOIN statistics s
ORDER BY r.timestamp;
```

This query produces a dataset containing both the minute-level rates and the statistical thresholds derived from historical behavior.

4.4 Statistical Baseline Table

The statistical distribution of the monitored rates is summarized below.

Metric	Mean	Median	P95	P99	Max
deny_rate	5.58%	4.72%	10.42%	28.45%	40.35%
fail_rate	0.054%	0%	0%	2.08%	8.70%
reverse_rate	0.781%	0.758%	2.33%	3.25%	5.34%

These values represent the natural variability of the system and can be used to establish anomaly thresholds.

4.5 Monitoring Thresholds

Based on the observed distribution:

Denied transactions

Warning: deny_rate > 10%
Critical: deny_rate > 20%

Failed transactions

Warning: fail_rate > 1%
Critical: fail_rate > 3%

Reversed transactions

Warning: reverse_rate > 2%
Critical: reverse_rate > 3%

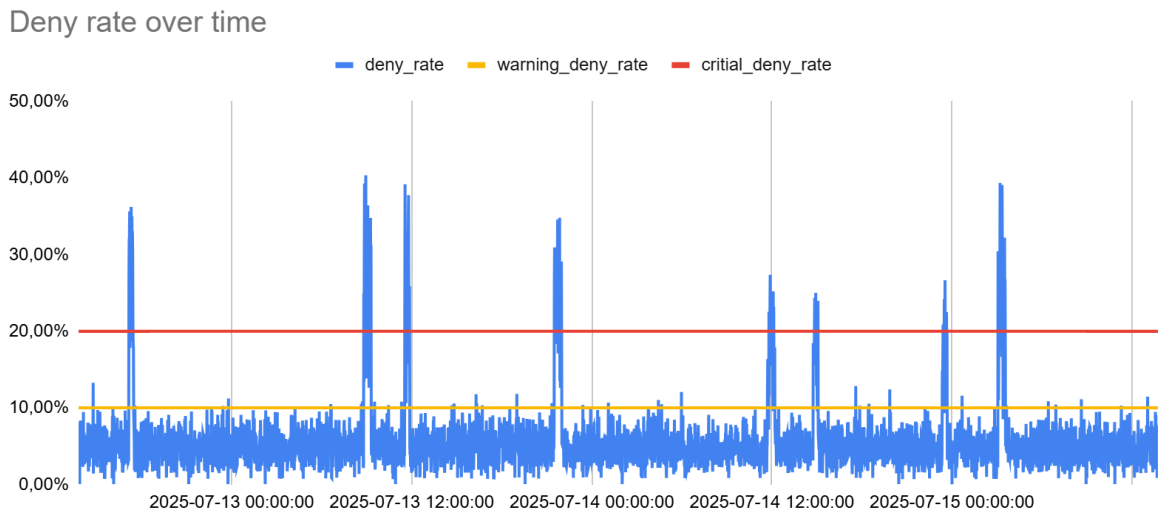
These thresholds correspond to behavior outside normal operating ranges.

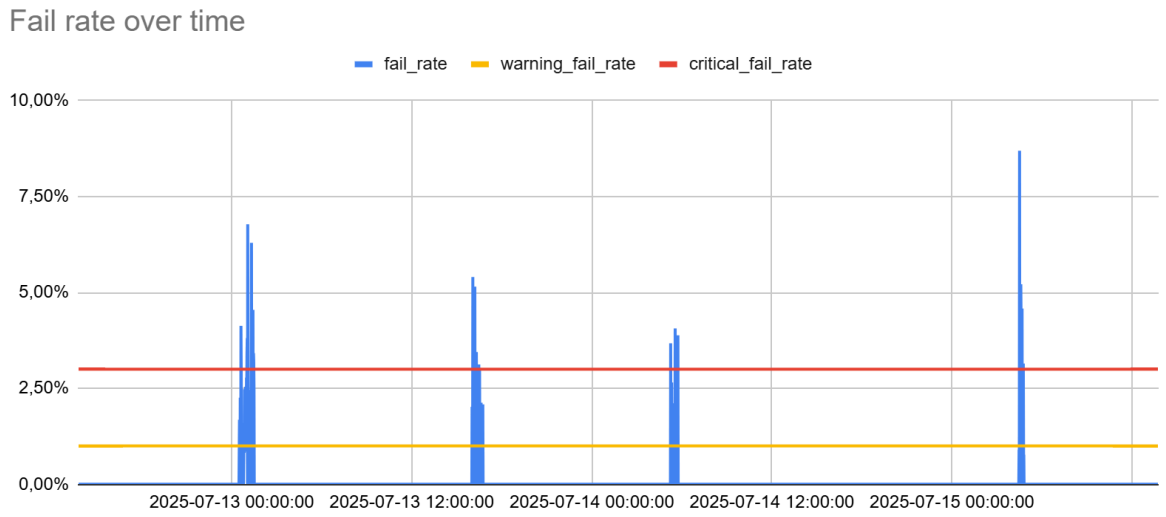
4.6 Visualization

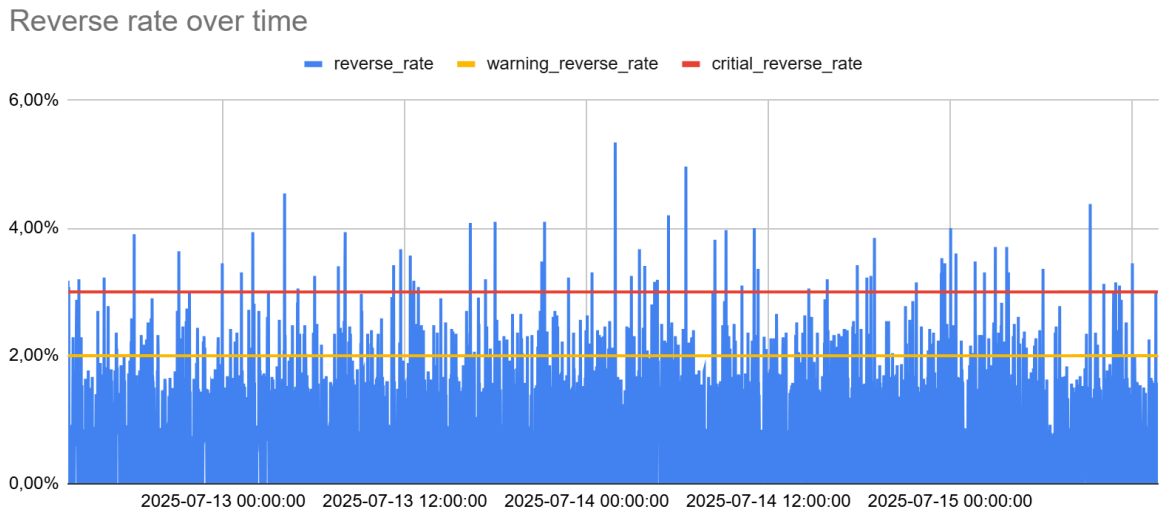
To support operational monitoring, three time-series visualizations were created in Google Sheets:

- deny_rate over time
- fail_rate over time
- reverse_rate over time

These graphs allow the identification of sustained deviations relative to expected behavior.







Visualizing the time-series behavior makes it easier to identify sustained anomalies that may not be visible in aggregated metrics.

5. Monitoring Endpoint Implementation (Python)

5.1 Design Rationale

The monitoring endpoint was designed to receive aggregated transaction counts per time window, typically one minute, as produced by the SQL aggregation query described in section 4.3.

Each payload represents a single observation window and contains the raw transaction counts required to compute the three monitored rates:

- deny_rate
- fail_rate
- reverse_rate

The thresholds applied in the alert logic are not arbitrary. They were derived directly from the statistical distribution of the `transactions.csv` dataset, as summarized in section 4.4.

Specifically:

- **Warning thresholds** correspond to behaviour near or above the **P95** of each rate distribution.
- **Critical thresholds** correspond to behaviour approaching or exceeding the **P99** boundary, representing the most extreme observations in the dataset.

This approach allows alert thresholds to reflect the natural behaviour of the system rather than relying on arbitrary limits.

5.2 Threshold Reference

The table below maps each threshold used by the monitoring logic to its statistical origin in the dataset.

Metric	Warning threshold	Critical threshold	Statistical basis
deny_rate	> 10%	> 20%	P95 = 10.42% / P99 = 28.45%
fail_rate	> 1%	> 3%	Max observed = 8.70% (rare spikes)

reverse_rate	> 2%	> 3%	P95 = 2.33% / P99 = 3.25%

For `fail_rate`, the P95 was effectively zero because failure events are rare in the dataset.

For this reason, a conservative operational threshold was selected to allow meaningful detection of abnormal spikes.

5.3 Implementation

The monitoring endpoint is implemented using **FastAPI** with **Pydantic schema validation**.

The core components are:

- **TransactionWindow**
  A typed input schema that validates the payload and prevents division by zero errors before rate calculation.
- **THRESHOLDS**
  A configuration dictionary that centralises all threshold values and documents their statistical origin through inline comments.
- **detect_anomaly()**
  A pure function that evaluates the monitored rates against their thresholds and returns the most severe alert classification found.
- **POST /monitor**
  The endpoint responsible for orchestrating input validation, rate calculation, anomaly detection, and response formatting.

```
from fastapi import FastAPI
from pydantic import BaseModel, field_validator
from datetime import datetime

app = FastAPI()

# Thresholds derived from statistical analysis of transactions.csv
# deny_rate:    P95 = 10.42% → WARNING | P99 = 28.45% → CRITICAL (rounded to 20%)
# fail_rate:    P95 = 0.00% → WARNING at 1% | Max observed = 8.70% → CRITICAL at 3%
# reverse_rate: P95 = 2.33% → WARNING at 2% | P99 = 3.25% → CRITICAL at 3%

THRESHOLDS = {
   "deny": {"warning": 0.10, "critical": 0.20},
   "fail": {"warning": 0.01, "critical": 0.03},
   "reverse": {"warning": 0.02, "critical": 0.03},
}

class TransactionWindow(BaseModel):
   timestamp: datetime | None = None
   total: int
```

```
    denied: int
    failed: int
    reversed: int

    @field_validator("total")
    @classmethod
    def total_must_be_positive(cls, v):
        if v <= 0:
            raise ValueError("total must be greater than zero")
        return v


def detect_anomaly(deny_rate: float, fail_rate: float, reverse_rate: float) -> str:
    rates = {"deny": deny_rate, "fail": fail_rate, "reverse": reverse_rate}

    for metric, value in rates.items():
        if value > THRESHOLDS[metric]["critical"]:
            return f"CRITICAL_{metric.upper()}"

    for metric, value in rates.items():
        if value > THRESHOLDS[metric]["warning"]:
            return f"WARNING_{metric.upper()}"

    return "NORMAL"


@app.post("/monitor")
def monitor_transaction(data: TransactionWindow):

    deny_rate = data.denied / data.total
    fail_rate = data.failed / data.total
    reverse_rate = data.reversed / data.total

    alert = detect_anomaly(deny_rate, fail_rate, reverse_rate)

    return {
        "timestamp": data.timestamp or datetime.utcnow(),
        "deny_rate": round(deny_rate, 4),
        "fail_rate": round(fail_rate, 4),
        "reverse_rate": round(reverse_rate, 4),
        "alert": alert,
    }
```

5.4 Request and Response Format

Example request payload (JSON):

```
{
 "timestamp": "2025-07-13T14:32:00",
 "total": 480,
 "denied": 62,
```

```
 "failed": 4,
 "reversed": 9
}
```

Expected response:

```
{
 "timestamp": "2025-07-13T14:32:00",
 "deny_rate": 0.1292,
 "fail_rate": 0.0083,
 "reverse_rate": 0.0188,
 "alert": "WARNING_DENY"
}
```

5.5 Integration with the SQL Pipeline

This endpoint is designed to operate as the final stage of the monitoring pipeline described in this document.

The expected data flow is the following:

1. The SQL query from section 4.3 runs at regular intervals, ideally once per minute, and aggregates raw transaction counts by status.
2. The aggregated result for each time window is serialised as a JSON payload and sent to `POST /monitor`.
3. The endpoint evaluates the transaction rates and returns an alert classification for that window.
4. Any response where `alert != NORMAL` can then be forwarded to the notification system for operator review.

This design keeps the anomaly detection logic decoupled from the data aggregation layer. As a result, it becomes straightforward to adjust thresholds, add new metrics, or replace the transport mechanism without modifying the SQL pipeline.

---

6. Final Conclusion

The checkout datasets revealed both temporary degradations and partial outages that would not be visible through daily aggregated metrics.

The transaction datasets showed that denial spikes represent the most significant anomaly pattern, while failure events are rare but highly sensitive indicators of system issues.

The monitoring system proposed combines statistical baselines, rate-based metrics, and rule-based alerting to detect abnormal behavior in real time.

This approach offers a simple and interpretable way to monitor transactional behavior and to highlight potential anomalies in the dataset.